

Executive Summary

This is the full plan for a **Arduino & AI Innovators** camp of **55 children** on 6th August 2026 at Fireside Group. The intake is split into age bands — 7-9 yrs (20), 10-12 yrs (23), 13-15 yrs (12) — and each band into small groups, so every child works at the right level and gets plenty of hands-on time. The schedule, staffing, topic index and full lesson content follow below.

55
CHILDREN

3
AGE BANDS

7
GROUPS

12
ADULTS

1:4.6
RATIO

Full Day
LENGTH

Learning outcomes

- Every child builds and tests real projects pitched at their age and level.
- Children work in small teams — sharing roles, debugging together and helping one another.
- Each group presents what they made at the end-of-day showcase, building confidence and communication.

Master Day Timetable

The whole camp runs on one clock. Groups start and stop together so breaks, lunch and the showcase stay in sync.

TIME	ACTIVITY
08:30 – 09:00	Arrival, registration & name badges
09:00 – 09:20	Opening assembly — welcome, safety brief, split into groups (all bands together)

TIME	ACTIVITY
09:20 – 10:45	Project Block 1 — each group in its own space
10:45 – 11:05	Morning break & snack (staggered so the space doesn't crowd)
11:05 – 12:30	Project Block 2 — build continues / next activity
12:30 – 13:15	Lunch & free play (supervised, outdoors if possible)
13:15 – 14:30	Project Block 3 — finish builds, test & debug
14:30 – 14:50	Showcase prep — groups tidy kits and rehearse a 60-second demo
14:50 – 15:20	Showcase & demos — every group presents to the whole camp
15:20 – 15:30	Closing circle, certificates & departure

Groups at a Glance

Each group stays together all day with its own lead facilitator, so no child is ever in a group larger than the target size.

AGE BAND	GROUP	SIZE
7-9 yrs	7-9 yrs · Group A	10 children
7-9 yrs	7-9 yrs · Group B	10 children
10-12 yrs	10-12 yrs · Group A	8 children
10-12 yrs	10-12 yrs · Group B	8 children
10-12 yrs	10-12 yrs · Group C	7 children
13-15 yrs	13-15 yrs · Group A	6 children
13-15 yrs	13-15 yrs · Group B	6 children

Staffing & Ratios

With **55 children** in **7 groups**, this plan needs **12 adults** on the day — an overall ratio of about **1 adult to 4.6 children** (target 1:10 or better).

ROLE	PEOPLE
Lead facilitators (one per group)	7

ROLE	PEOPLE
Floating helpers	3
Camp coordinator	1
First-aider	1
Total adults	12

Topics in This Plan

Each band's topics are listed by number below. A topic run by more than one band appears once in the programme (with its bands noted), so its instructions are never repeated — the same number just shows up for each band that runs it.

AGE BAND	TOPICS (BY NUMBER)
7-9 yrs 20 children	1. Traffic light simulation (<i>Beginner</i> · ~60 min) 2. Basic alarm system (<i>Intermediate</i> · ~90 min) 3. Smart city (<i>Advanced</i> · ~120 min) 4. Teachable Machine (<i>Intermediate</i> · ~90 min) 5. AI art studio (<i>Beginner</i> · ~60 min)
10-12 yrs 23 children	1. Traffic light simulation (<i>Beginner</i> · ~60 min) 2. Basic alarm system (<i>Intermediate</i> · ~90 min) 3. Smart city (<i>Advanced</i> · ~120 min) 4. Teachable Machine (<i>Intermediate</i> · ~90 min) 5. AI art studio (<i>Beginner</i> · ~60 min)
13-15 yrs 12 children	1. Traffic light simulation (<i>Beginner</i> · ~60 min) 2. Basic alarm system (<i>Intermediate</i> · ~90 min) 3. Smart city (<i>Advanced</i> · ~120 min) 4. Teachable Machine (<i>Intermediate</i> · ~90 min) 5. AI art studio (<i>Beginner</i> · ~60 min)

Lesson Topics

Each topic below is a self-contained card with its materials, step-by-step instructions and full code/build steps. Where a topic is run by more than one age band, it is shown **once** and the bands that run it are listed on the card — the instructions are the same for each, so there is no need to repeat them.

1 Traffic light simulation

BEGINNER · ~60 MIN · BANDS: 7-9 YRS (20), 10-12 YRS (23), 13-15 YRS (12) · 7 GROUP(S)

MATERIALS

- 1 x Arduino Uno board (Nerokas.co.ke)
- 1 x USB cable (Type A to Type B) for programming (Nerokas.co.ke)
- 1 x Breadboard, half-size or full-size (Pixel Electronics.co.ke)
- 1 x Red LED, 5mm (Nerokas.co.ke)
- 1 x Yellow (or amber) LED, 5mm (Nerokas.co.ke)
- 1 x Green LED, 5mm (Nerokas.co.ke)
- 3 x 220 ohm resistors (colour bands: red-red-brown) (Pixel Electronics.co.ke)
- 4 x Male-to-male jumper wires (Nerokas.co.ke)
- 1 x Laptop or computer with Arduino IDE installed (not purchased locally)

HOW IT WORKS

Real traffic lights follow a fixed pattern: red means stop, green means go, and yellow (or amber) warns drivers that the light is about to change. This project recreates that pattern using three LEDs (Light Emitting Diodes) — one red, one yellow, one green — controlled by an Arduino.

An Arduino is a small computer that can turn electricity on and off very precisely, using a program (called a "sketch") that we write and upload to it. Each LED is connected to a different pin on the Arduino. In our code, we tell the Arduino things like "turn on the red LED, wait 5 seconds, turn it off, turn on the green LED..." and so on, in a loop that repeats forever.

LEDs only allow electricity to flow in one direction and they need just the right amount of current — too much and they burn out instantly. That is why each LED is paired with a 220 ohm resistor, which acts like a narrow gate that limits how much electricity can flow through, protecting the LED from damage.

The core programming idea here is **sequencing with timing**: the Arduino executes instructions one after another, and the `delay()` function tells it to pause for a set number of milliseconds before moving to the next instruction. By arranging `digitalWrite()` (turn a pin HIGH or LOW) and `delay()` commands in the right order inside a loop, we recreate the real-world timing of a traffic light.

STEP-BY-STEP INSTRUCTIONS

1. Place the breadboard in front of you with the long power rails running along the top and bottom edges.
2. Insert the red LED into the breadboard. Note that LEDs have one longer leg (the anode, positive) and one shorter leg (the cathode, negative). Place it so its two legs are in different rows of the breadboard.
3. Repeat for the yellow and green LEDs, spacing them out so each has its own set of rows and does not touch its neighbours.
4. For each LED, connect a 220 ohm resistor from the LED's shorter leg (cathode) row to the breadboard's ground (-) rail.
5. Run one jumper wire from the ground (-) rail on the breadboard to the GND pin on the Arduino.
6. Run a jumper wire from the red LED's longer leg (anode) row to digital pin 13 on the Arduino.
7. Run a jumper wire from the yellow LED's longer leg (anode) row to digital pin 12 on the Arduino.
8. Run a jumper wire from the green LED's longer leg (anode) row to digital pin 11 on the Arduino.
9. Double-check that no bare wires are touching each other on the breadboard, and that each LED has its own resistor in its path.
10. Connect the Arduino to the computer using the USB cable.
11. Open the Arduino IDE, select the correct board (Arduino Uno) and port from the Tools menu.
12. Copy the sketch from the "Full Code" section into a new sketch window.
13. Click the Upload button (right-facing arrow icon) and wait for "Done uploading" to appear.
14. Observe: the red LED should light for several seconds, then turn off as green lights up, then green turns off as yellow blinks briefly, then it returns to red — repeating in a continuous loop.
15. If an LED does not light, check that it is inserted the correct way round (long leg towards the Arduino pin side) before assuming the code is wrong.
16. Once working, ask students to time the sequence with a stopwatch and compare it to real traffic lights they have seen.

FULL CODE

```
// Traffic Light Simulation
// Three LEDs (red, yellow, green) mimic a real traffic light sequence

// Assign each LED colour to an Arduino digital pin
const int redPin = 13;    // Red LED connected to pin 13
const int yellowPin = 12; // Yellow LED connected to pin 12
const int greenPin = 11;  // Green LED connected to pin 11

// Timing settings (in milliseconds) - easy to change later
const int redDuration = 4000;    // Red light stays on for 4 seconds
const int greenDuration = 4000;  // Green light stays on for 4 seconds
const int yellowDuration = 1000; // Yellow light stays on for 1 second

void setup() {
  // Set all three LED pins as OUTPUT so the Arduino can send power to them
  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
}

void loop() {
  // --- RED PHASE: Stop ---
  digitalWrite(redPin, HIGH);    // Turn red LED on
  digitalWrite(yellowPin, LOW);  // Make sure yellow is off
  digitalWrite(greenPin, LOW);   // Make sure green is off
  delay(redDuration);            // Wait while red stays lit

  // --- GREEN PHASE: Go ---
  digitalWrite(redPin, LOW);     // Turn red off
  digitalWrite(greenPin, HIGH);  // Turn green on
  delay(greenDuration);          // Wait while green stays lit

  // --- YELLOW PHASE: Prepare to stop ---
  digitalWrite(greenPin, LOW);   // Turn green off
  digitalWrite(yellowPin, HIGH); // Turn yellow on as a warning
  delay(yellowDuration);         // Wait a short time on yellow
  digitalWrite(yellowPin, LOW);  // Turn yellow off before looping back to red

  // The loop() function repeats automatically, so it goes back to RED PHASE
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
An LED does not light up at all	LED inserted backwards (legs reversed)	Remove the LED, flip it around so the longer leg (anode) connects towards the Arduino pin, and reinsert
LED glows very faintly or flickers	Loose jumper wire or resistor not fully pushed into breadboard holes	Press all wires and resistor legs firmly into the breadboard rows

All LEDs stay off after uploading	Wrong board or port selected in Arduino IDE, or upload failed silently	Check Tools > Board and Tools > Port, then re-upload and wait for "Done uploading" message
Two LEDs light up at the same time when they should not	<code>digitalWrite(..., LOW)</code> line missing or commented out for the previous colour	Check each phase in the code turns off the LED(s) that should no longer be lit
Sequence timing looks wrong (too fast or too slow)	<code>delay()</code> values not matched to the intended real-world timing	Adjust the <code>redDuration</code> , <code>greenDuration</code> and <code>yellowDuration</code> constants at the top of the sketch
LED burns out (goes dark permanently, sometimes with a small pop or smell)	Missing or wrong-value resistor allowed too much current through the LED	Always use the 220 ohm resistor in series with each LED before power is applied

DISCUSSION QUESTIONS

- Why do you think traffic lights use red, yellow, and green specifically, rather than any other three colours?
- What would happen in our code if we forgot to turn off the red LED before turning on the green one?
- Why does the resistor need to be placed in the circuit, and what might happen to the LED without it?
- In the code, what is the purpose of the `delay()` function, and what do you think would happen if all the delay values were changed to 100 milliseconds?
- Real traffic lights sometimes have a flashing yellow warning mode at night. How could you modify this program to add that behaviour?

EXTENSION CHALLENGES

- **Easy:** Change the timing so red lasts 6 seconds and green lasts 3 seconds, then re-upload and observe the difference.
- **Medium:** Add a push button that, when pressed, immediately switches the light to red (like a pedestrian crossing button), then resumes the normal sequence afterward.
- **Hard:** Build a second traffic light for a perpendicular road using three more LEDs, and write code so that when one light is green, the other is red, simulating a real road intersection.

2 Basic alarm system

INTERMEDIATE · ~90 MIN · BANDS: 7-9 YRS (20), 10-12 YRS (23), 13-15 YRS (12) · 7 GROUP(S)

MATERIALS

- 1 x Arduino Uno R3 (Nerokas.co.ke)
- 1 x PIR Motion Sensor Module (HC-SR501) (Pixel Electronics - pixelelectronics.co.ke)
- 1 x Active Buzzer Module (5V) (Nerokas.co.ke)
- 1 x Red LED (5mm) (Nerokas.co.ke)
- 1 x Green LED (5mm) (Pixel Electronics - pixelelectronics.co.ke)
- 2 x 220 ohm Resistors (for LEDs) (Nerokas.co.ke)
- 1 x Breadboard (830 tie points) (Pixel Electronics - pixelelectronics.co.ke)
- 1 x Push Button (for arming/disarming the alarm) (Nerokas.co.ke)
- 1 x 10k ohm Resistor (for push button pull-down) (Nerokas.co.ke)
- Male-to-Male Jumper Wires (pack of at least 15) (Pixel Electronics - pixelelectronics.co.ke)
- 1 x USB Cable (Type A to Type B) for Arduino Uno (Nerokas.co.ke)
- 1 x 9V Battery with Arduino Barrel Connector (optional, for portable demo) (Pixel Electronics - pixelelectronics.co.ke)

HOW IT WORKS

Imagine you have a security guard whose only job is to watch a room and shout if someone walks in who shouldn't be there. In this project, our "guard" is a small sensor called a PIR (Passive Infrared) sensor. This sensor can detect the invisible heat that all living things give off. When a warm body (like a person) moves in front of it, the sensor notices the change in heat and sends a signal to the Arduino saying "Something moved!"

The Arduino is like the guard's brain. It is constantly checking the PIR sensor's signal, over and over, many times per second. This is called **polling** — repeatedly asking "did anything change yet?" When the Arduino sees the "motion detected" signal, it makes a decision in the code (this is called a **conditional** — an "if this happens, then do that" instruction) and reacts by turning on a red LED and sounding a buzzer, just like a real alarm.

We also add a push button that acts as a key switch — pressing it once "arms" the alarm (turns it on and ready), and pressing it again "disarms" it (turns it off), just like you would with a real house alarm keypad. A green LED shows when the system is safely armed and watching.

This project combines three important ideas: **sensing** (the PIR detecting motion), **decision-making** (the Arduino code deciding what to do with that information), and **output** (the buzzer and LEDs reacting). This is exactly how real-world security systems, streetlights, and even automatic doors work.

STEP-BY-STEP INSTRUCTIONS

1. Lay the breadboard flat on the table and place the Arduino next to it, not touching.
2. Insert the PIR sensor's three pins into the breadboard (or connect directly with jumper wires if it has a pin header cable). Identify the VCC, OUT, and GND pins — they are usually labelled on the small circuit board.
3. Connect the PIR sensor's VCC pin to the Arduino's 5V pin using a jumper wire.
4. Connect the PIR sensor's GND pin to the Arduino's GND pin using a jumper wire.
5. Connect the PIR sensor's OUT pin to Arduino digital pin 2 using a jumper wire.
6. Place the red LED on the breadboard. Connect its longer leg (anode) through a 220 ohm resistor to Arduino digital pin 8. Connect its shorter leg (cathode) to the breadboard's GND rail, then to Arduino GND.
7. Place the green LED on the breadboard the same way, but connect its resistor-anode side to Arduino digital pin 9. Connect its cathode to GND.
8. Connect the buzzer's positive pin to Arduino digital pin 7, and its negative pin to GND.
9. Set up the push button: connect one leg to Arduino digital pin 4, and the same leg through a 10k ohm resistor to GND (this is the pull-down resistor, keeping the reading steady when the button is not pressed). Connect the opposite leg of the button to 5V.
10. Double-check all GND connections are shared on a common rail — a loose ground wire is the most common wiring mistake.
11. Connect the Arduino to the computer with the USB cable and open the Arduino IDE.

12. Type or paste the code from the Full Code section below, then upload it to the Arduino.
13. Open the Serial Monitor (set to 9600 baud) to watch status messages as you test the system.
14. Press the push button once. Observe the green LED turn on — this means the alarm is now "armed."
15. Wait about 15-20 seconds without moving — PIR sensors need a short warm-up/calibration time when first powered.
16. Wave your hand or walk in front of the PIR sensor. Observe the red LED turn on and the buzzer sound — this is the alarm triggering.
17. Press the push button again to disarm the system. Observe the green LED turn off, and confirm the buzzer stops responding to motion.
18. Let students take turns arming, triggering, and disarming the system while explaining out loud what they think is happening at each step.

FULL CODE

```
// ----- Basic Alarm System -----
// Motion sensor triggers buzzer & LED alarm
// Push button arms/disarms the system

const int pirPin = 2;      // PIR sensor OUTPUT pin
const int buzzerPin = 7;   // Buzzer signal pin
const int redLedPin = 8;   // Red LED = alarm triggered
const int greenLedPin = 9; // Green LED = system armed
const int buttonPin = 4;   // Push button pin

bool systemArmed = false;   // Tracks whether alarm is currently armed
bool lastButtonState = LOW; // Stores previous button reading to detect a new press
bool alarmTriggered = false; // Tracks whether alarm is currently sounding

void setup() {
  pinMode(pirPin, INPUT);      // PIR sends HIGH/LOW signal, so it's an input
  pinMode(buzzerPin, OUTPUT);  // We control the buzzer, so it's an output
  pinMode(redLedPin, OUTPUT);  // We control the red LED
  pinMode(greenLedPin, OUTPUT); // We control the green LED
  pinMode(buttonPin, INPUT);   // Button reading comes into the Arduino

  Serial.begin(9600); // Start Serial communication so we can see debug messages
  Serial.println("System ready. Press button to arm the alarm.");

  delay(15000); // Give the PIR sensor time to calibrate to the room's normal heat levels
  Serial.println("PIR sensor calibrated and ready.");
}

void loop() {
  bool currentButtonState = digitalRead(buttonPin); // Read the button right now
```

```

// Detect a NEW press (button just went from LOW to HIGH)
if (currentButtonState == HIGH && lastButtonState == LOW) {
  systemArmed = !systemArmed; // Flip the armed status every time button is newly pressed
  Serial.println(systemArmed ? "System ARMED" : "System DISARMED");
  delay(50); // Small delay to avoid reading the same press twice (debounce)
}

lastButtonState = currentButtonState; // Save this reading for next loop comparison

digitalWrite(greenLedPin, systemArmed ? HIGH : LOW); // Green LED shows armed status

if (systemArmed) {
  int motionDetected = digitalRead(pirPin); // Check what the PIR sensor sees

  if (motionDetected == HIGH) {
    alarmTriggered = true; // Motion found while armed = trigger the alarm
  }

  if (alarmTriggered) {
    digitalWrite(redLedPin, HIGH); // Turn on red LED
    digitalWrite(buzzerPin, HIGH); // Turn on buzzer sound
    Serial.println("ALARM! Motion detected!");
  }
} else {
  // If system is disarmed, make sure alarm outputs are off
  alarmTriggered = false;
  digitalWrite(redLedPin, LOW);
  digitalWrite(buzzerPin, LOW);
}

delay(100); // Short pause to keep loop from running too fast and flooding Serial Monitor
}

```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
Alarm triggers constantly, even with no movement	PIR sensor was not given enough calibration/warm-up time, or its sensitivity potentiometer is set too high	Wait the full 15-20 seconds after power-up before testing; turn the sensitivity dial on the PIR module counter-clockwise slightly
Nothing happens at all — no LEDs, no buzzer, no Serial messages	Loose USB connection or code did not upload successfully	Check the Arduino IDE for an upload error message; reconnect USB cable; confirm correct board and port are selected
Button press doesn't arm/disarm the system, or arms and disarms randomly	Missing pull-down resistor causing "floating" voltage readings on the button pin	Confirm the 10k ohm resistor is correctly wired between the button pin and GND
LEDs stay dim or don't light up fully	LED wired backwards (reversed polarity) or missing/wrong resistor value	Flip the LED around so the longer leg (anode) connects to the resistor/pin side; confirm 220 ohm resistor is used

Buzzer stays silent even though red LED lights up	Buzzer wired to wrong pin, or using a passive buzzer that needs a <code>tone()</code> signal instead of HIGH/LOW	Confirm buzzer type is "active" (usually has a sticker/dome top); check wiring matches pin 7 in the code
Alarm won't turn off even after disarming with the button	<code>alarmTriggered</code> variable not resetting because of a logic error in code	Confirm the <code>else</code> block correctly sets <code>alarmTriggered = false</code> when <code>systemArmed</code> is false

DISCUSSION QUESTIONS

- What does the PIR sensor actually detect — is it seeing light, sound, or something else?
- Why did we need to wait 15-20 seconds before testing the alarm? What might happen if we skipped that wait?
- What is the purpose of the pull-down resistor on the push button? What do you think would happen if we removed it?
- Can you think of a real place in your home, school, or neighbourhood where a system like this could actually be useful?
- If you wanted the alarm to also send a message to someone's phone, what extra parts or ideas do you think you'd need?

EXTENSION CHALLENGES

- **Easy:** Add a second buzzer tone pattern (using `tone()` and `noTone()`) so the alarm makes a beeping sound instead of a constant tone.
- **Medium:** Add a delay so that once armed, the system waits 10 seconds before actually starting to watch for motion (giving the "owner" time to leave the room, like a real house alarm).
- **Hard:** Add an LCD screen or use the Serial Monitor to display a live countdown timer, an "entry delay" before the alarm sounds, and a running log of how many times the alarm has been triggered since the Arduino was powered on.

3 Smart city

ADVANCED · ~120 MIN · BANDS: 7-9 YRS (20), 10-12 YRS (23), 13-15 YRS (12) · 7 GROUP(S)

OVERVIEW

The Smart City build brings three city subsystems onto one Arduino: a **traffic-light intersection**, an **automatic street light** that turns on when it gets dark, and a **smart-parking bay** that senses whether a car is present and shows the status on

an LCD. It is the flagship "Solve" project — students integrate several sensors and outputs into one working model of their town.

MATERIALS

- Arduino Uno & USB cable — 1 (Nerokas, nerokas.co.ke)
- Breadboard & jumper wires — 1 set (Nerokas / Pixel Electronics, pixelelectronics.co.ke)
- LEDs: red, yellow, green for traffic + 1 for street light + 2 for parking status — 6 total, with 220Ω resistors
- LDR (light-dependent resistor) + 10kΩ resistor — 1 (voltage divider into A0)
- HC-SR04 ultrasonic sensor — 1 (parking)
- 16x2 LCD (parallel / HD44780) + 10kΩ potentiometer for contrast — 1

Optional for extensions: DHT11, MQ135 air-quality, PIR motion, soil-moisture, servo, buzzer, relay, ESP8266/ESP32 for IoT.

HOW IT WORKS

- **Traffic lights** cycle green → yellow → red on a timer written as a non-blocking state machine (using `millis()`), so the rest of the city keeps running.
- **Street light** reads the LDR each loop; when the reading drops below the "dark" threshold the LED switches on — just like real dusk-to-dawn lighting.
- **Smart parking** pings the ultrasonic sensor: a car within 10 cm marks the bay *occupied* (red LED), otherwise *free* (green LED), and the LCD shows FREE / FULL.

WIRING & PIN MAP

PART	ARDUINO PIN(S)
Traffic red / yellow / green	13 / 12 / 11
Street-light LED	10
Parking free / occupied LED	7 / 6
Ultrasonic TRIG / ECHO	9 / 8
LDR divider	A0
LCD rs, en, d4, d5, d6, d7	2, 3, 4, 5, A1, A2

STEP-BY-STEP BUILD

1. Build the traffic light first (3 LEDs + resistors) and get the timed sequence running.
2. Add the LDR as a voltage divider into A0; add the street-light LED and test dark/light.
3. Wire the HC-SR04 (TRIG 9, ECHO 8) and the two parking LEDs; test the distance reading on the Serial Monitor.
4. Wire the LCD (contrast pot to the middle pin) and show the parking status on the bottom row.
5. Upload the full sketch below and tune `DARK` and `PARK_CM` for your room.

FULL CODE

```
#include

// 16x2 LCD in 4-bit mode: rs, en, d4, d5, d6, d7
LiquidCrystal lcd(2, 3, 4, 5, A1, A2);

// --- Pin map ---
const int RED = 13, YEL = 12, GRN = 11;    // traffic light
const int STREET = 10;                    // smart street light LED
const int TRIG = 9, ECHO = 8;              // parking ultrasonic (HC-SR04)
const int P_FREE = 7, P_OCC = 6;          // parking status LEDs
const int LDR = A0;                       // light sensor (analog)

// --- Tuning ---
const unsigned long GREEN_MS = 5000, YELLOW_MS = 2000, RED_MS = 5000;
const int DARK = 400;                    // LDR reading below this = "night" (tune on the day)
const int PARK_CM = 10;                  // car within 10 cm = bay occupied

int phase = 0;                          // 0 = green, 1 = yellow, 2 = red
unsigned long phaseStart = 0;
unsigned long lastSense = 0;

void setup() {
  int outs[] = {RED, YEL, GRN, STREET, P_FREE, P_OCC};
  for (int i = 0; i < 6; i++) pinMode(outs[i], OUTPUT);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  lcd.begin(16, 2);
  lcd.print("Smart City");
  phaseStart = millis();
}

// Read distance in cm; returns -1 on timeout (nothing in range).
long readCm() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 30000);    // 30 ms timeout
  return us == 0 ? -1 : us / 58;           // ~58 us per cm (round trip)
}
```

```
// Non-blocking traffic light so parking + lights keep updating.
void updateTraffic() {
  unsigned long now = millis();
  unsigned long dur = (phase == 0) ? GREEN_MS : (phase == 1) ? YELLOW_MS : RED_MS;
  if (now - phaseStart >= dur) { phase = (phase + 1) % 3; phaseStart = now; }
  digitalWrite(GRN, phase == 0);
  digitalWrite(YEL, phase == 1);
  digitalWrite(RED, phase == 2);
}

void loop() {
  updateTraffic();

  // Smart street light: switch on automatically when it gets dark.
  digitalWrite(STREET, analogRead(LDR) < DARK ? HIGH : LOW);

  // Smart parking: check the bay twice a second.
  if (millis() - lastSense >= 500) {
    lastSense = millis();
    long cm = readCm();
    bool occupied = (cm > 0 && cm <= PARK_CM);
    digitalWrite(P_OCC, occupied);
    digitalWrite(P_FREE, !occupied);
    lcd.setCursor(0, 1);
    lcd.print(occupied ? "Parking: FULL " : "Parking: FREE ");
  }
}
```

COMMON MISTAKES & FIXES

SYMPTOM	LIKELY CAUSE	FIX
Everything freezes during the red light	Using <code>delay()</code> for the traffic light	Use the <code>millis()</code> state machine shown here so sensors keep reading
Street light never turns on/off	<code>DARK</code> threshold wrong for the room	Print <code>analogRead(LDR)</code> to Serial and set <code>DARK</code> between the light and dark values
Parking always reads FULL or FREE	ECHO/TRIG swapped, or timeout	Check wiring; <code>readCm()</code> returns -1 on timeout — treat as "free"
LCD shows blocks or nothing	Contrast pot not set / miswired	Turn the 10k pot until text appears; recheck rs/en/d4-d7

DISCUSSION QUESTIONS

1. Why is a non-blocking (`millis()`) traffic light better than one built with `delay()`?
2. What real problem in your town could each subsystem help with?
3. How could the parking bay count several spaces instead of one?
4. Where would you add a sensor to make the city safer or greener?
5. What are the trade-offs of automating street lights (cost, safety, energy)?

EXTENSION CHALLENGES

- **Easy:** add a second intersection direction; count multiple free parking bays on the LCD.
- **Medium:** add an air-quality monitor (MQ135), a weather station (DHT11), automatic pedestrian lighting (PIR), or a home-security intruder alarm (PIR + buzzer).
- **Hard:** make the traffic lights respond to traffic volume (ultrasonic), add smart irrigation (soil-moisture + relay), or connect to the internet with an ESP8266/ESP32 and plot temperature, traffic and air quality on an Arduino Cloud / Blynk / ThingSpeak dashboard.
- **Capstone:** teams combine subsystems into one miniature city — traffic, smart homes, street lights, parking, environment sensors, an emergency system and an IoT dashboard.

12-WEEK CLUB PATHWAY (OPTIONAL)

For clubs and camps that want to run Smart City as a course, this build is the spine of a 12-week pathway:

WEEK	DELIVERABLE
1	Smart city plan (design thinking)
2	Blink LED — Arduino basics
3	Traffic lights
4	Smart intersection
5	Parking system
6	Weather station
7	Air-quality monitor
8	Automatic street lights
9	Smart home
10	Smart farm
11	IoT dashboard
12	Complete smart city demo

4 Teachable Machine

INTERMEDIATE · ~90 MIN · BANDS: 7-9 YRS (20), 10-12 YRS (23), 13-15 YRS (12) · 7 GROUP(S)

WHAT WE'RE DOING

In this hands-on activity, students will build their very own image classifier using Google's Teachable Machine — a free browser-based tool that lets anyone train a real machine learning model without writing a single line of code. Students will use their device's webcam (or uploaded photos) to teach a model to recognise different categories of images, such as hand gestures, objects, or facial expressions.

By the end of the session, students will have a working AI model that can correctly identify what it "sees" in real time, and — more importantly — they will understand the core concepts behind how machine learning actually works: collecting data, training a model, testing its accuracy, and recognising bias when the model gets things wrong. This is a foundational lesson for understanding that AI systems are only as good as the data used to train them.

This is an intermediate-level session, so it assumes students already have basic familiarity with what AI is and have perhaps seen a demo of image recognition before. Here, they move from "watching AI work" to "building an AI model themselves."

TOOLS & RESOURCES

- **Google Teachable Machine** — <https://teachablemachine.withgoogle.com/> (free, no login required, runs entirely in the browser; no data leaves the device unless exported)
- **A device with a webcam** — laptop, tablet, or desktop with a working camera. If no webcam is available, students can upload image files instead.
- **Chrome or Edge browser** (recommended for best camera permission handling; Teachable Machine may behave inconsistently on older browsers)
- **Optional: sample image folders** pre-downloaded on a USB drive or shared folder, in case internet bandwidth is limited and students need to upload images instead of using live webcam capture
- **Free-tier limits to watch for:** Teachable Machine is completely free with no account needed, but it runs entirely client-side, meaning large image classes or many samples can slow down or crash older/low-spec devices. Models are also not saved automatically to the cloud unless the student explicitly exports/

downloads or uploads to Google Drive — remind students to save their work before closing the tab.

STEP-BY-STEP ACTIVITY

1. Open a browser and navigate to teachablemachine.withgoogle.com. Click "Get Started," then choose "Image Project," and select "Standard image model."
2. Explain the interface: there are "Class" boxes on the left (where training images go), a "Training" panel in the middle, and a "Preview" panel on the right (where the live model will make predictions).
3. Rename Class 1 and Class 2 to something meaningful — for example, "Thumbs Up" and "Thumbs Down," or "Book" and "Phone." Add a third class ("Nothing" or "Background") to help the model learn what "neither category" looks like.
4. Click the webcam icon under each class box and grant camera permission when prompted by the browser.
5. For each class, hold down the "Hold to Record" button to capture at least 50–100 images while varying angle, distance, lighting, and background. Repeat for every class.
6. Click the "Train Model" button in the centre panel. Do not close the tab or switch away — training takes anywhere from 10 seconds to a few minutes depending on the number of images and device speed.
7. Once training finishes, test the model live in the "Preview" panel by holding up the object/gesture in front of the webcam and watching the confidence percentages update in real time.
8. Deliberately test with a "confusing" input — for example, an object that overlaps two classes or unusual lighting — and discuss why the model gets it wrong or right.
9. Go back and add 20–30 more training images to the class that performed worst, focusing on varied conditions, then retrain and re-test to observe improvement.
10. Export the model: click "Export Model," choose "Tensorflow.js," and either copy the shareable link or download the model files to keep a record of the finished project.
11. Have each student (or pair) present their trained model to the class for 1 minute, showing what it classifies and one thing they discovered while training it.

EXAMPLE OUTPUTS

- **Gesture Classifier:** A model trained on "Thumbs Up," "Thumbs Down," and "Open Palm" that correctly identifies each gesture with over 90% confidence, including when tested from different angles.
- **Object Sorter:** A model that distinguishes between a "Water Bottle," "Notebook," and "Phone" placed on a desk, demonstrating that the model has learned shape and colour cues rather than just background context.
- **Expression Detector:** A model trained on "Smiling Face," "Serious Face," and "Surprised Face" that reveals limitations — for instance, performing well on the student who trained it but poorly on a classmate with different facial features, sparking rich discussion on bias in training data.

FACILITATION TIPS

- Do a live demo first before letting students loose — show one full cycle (collect → train → test) so they see the whole workflow before diving in themselves.
- Encourage variety early: many students will collect all their training images sitting in the exact same position, which produces a model that fails the moment lighting or angle changes. Prompt them mid-activity to "move around, change the background, get closer and farther."
- Watch device performance — if a laptop is old or many browser tabs are open, training can be slow. Suggest closing other tabs before starting.
- Some students will get suspiciously perfect (100%) results initially — use this as a teaching moment: ask "would this still work if I stood by the window?" and test it live to reveal overfitting.
- Camera permission pop-ups are a common stumbling block — walk the room early to make sure everyone has granted access before the training phase begins.
- Keep training sample counts roughly balanced across classes; a common confusion is students recording 200 images for one class and 20 for another, which skews the model's predictions toward the larger class.
- Plan for roughly: 10 min setup/demo, 15 min class naming and data collection, 10 min first training and testing, 15 min data improvement and retraining, 15 min export/sharing, remaining time for presentations and discussion.
- If webcams are unavailable or unreliable, have a backup folder of pre-selected images ready so students can use the "upload" option instead of live capture.

DISCUSSION QUESTIONS

- What steps did the model go through between you showing it images and it being able to make predictions?
- Why did the model perform differently when you changed the lighting, angle, or background — what does this tell you about how it "sees"?
- If someone else used your trained model, do you think it would work as well for them as it did for you? Why or why not?
- What is the difference between the model "memorising" your exact training images versus actually "learning" the category?
- Where in daily life might a similar image classifier already be working behind the scenes — and what could go wrong if its training data was unfair or incomplete?

EXTENSION CHALLENGES

- **Easy:** Add a fourth class to your existing model and retrain it — test whether adding more categories makes the model less accurate overall, and explain why.
- **Medium:** Deliberately try to "break" your model by testing it with tricky inputs (poor lighting, partial objects, mixed backgrounds). Document at least three failure cases and propose a specific fix (e.g., more images, different angles) for each one.
- **Hard:** Export your trained model's shareable link and swap with a partner. Test your partner's model using your own face/objects instead of theirs, record the accuracy drop, and write a short explanation of why the model behaves differently for a "new" user — connecting this to real-world concerns about AI systems trained on narrow or non-representative datasets.

5 AI art studio

BEGINNER · ~60 MIN · BANDS: 7-9 YRS (20), 10-12 YRS (23), 13-15 YRS (12) · 7 GROUP(S)

WHAT WE'RE DOING

In this workshop, students step into the role of "AI art directors" — they will use DALL-E, a text-to-image AI tool, to turn written descriptions (called prompts) into original pictures. Rather than drawing with a pencil, students will "draw with words," learning how small changes in language can dramatically change an image. By the end of the session, each student will have created a small personal gallery of AI-generated artworks and will understand, at a basic level, how AI image generators interpret text and turn it into visuals.

The activity is designed to build AI literacy through hands-on experimentation: students will notice patterns in what works and what doesn't, discover the AI's strengths and quirks, and reflect on questions of creativity, authorship, and bias in AI-generated art.

TOOLS & RESOURCES

- **DALL-E (via ChatGPT with image generation):** <https://chat.openai.com> — Free accounts have a limited number of image generations per day/hour; this can reset or run out mid-lesson, so plan accordingly.
- **Bing Image Creator (powered by DALL-E):** <https://www.bing.com/images/create> — Free with a Microsoft account; uses a "boost" credit system that limits fast generations, but images can still be created (slower) after credits run out.
- **Craiyon (free, no login):** <https://www.craiyon.com> — Good backup tool if internet is slow or accounts are unavailable; lower image quality but no sign-up needed.
- **A shared document or slide deck** (Google Slides/Docs) for students to paste their prompts and screenshot their images for later review.
- **Projector or shared screen** for teacher demonstration before students work individually or in pairs.

STEP-BY-STEP ACTIVITY

1. Open the chosen AI art tool (e.g., Bing Image Creator or ChatGPT) on the projector and explain: "This is an AI that turns your written descriptions into pictures. We call these descriptions 'prompts.'"
2. Demonstrate with a simple prompt: *"A red bicycle parked under a mango tree"*. Generate the image live and discuss what the AI got right or wrong.
3. Show how adding detail changes the result. Regenerate using: *"A red bicycle parked under a mango tree, in the style of a watercolour painting, warm afternoon light"*. Ask students to compare the two outputs.
4. Introduce the "prompt formula": **Subject + Setting + Style + Mood/ Lighting**. Write this formula somewhere visible for reference.
5. Have students log in (individually or in pairs) to their assigned tool.
6. Give students their first guided prompt to try: *"A lion wearing sunglasses, sitting on a rooftop in Nairobi, cartoon style, sunny day"*. Let them generate and view the result.

7. Ask students to write their own original prompt using the prompt formula, generate the image, and save/screenshot it.
8. Have students generate at least 2 variations of their original prompt (changing style, mood, or setting) to see how the output changes.
9. Ask each student to choose their favourite image and write one sentence explaining why the AI's interpretation surprised them or matched their expectations.
10. Bring the group back together to share 3–4 favourite images on the projector, reading the prompts aloud before revealing the image.
11. Close with a group discussion using the questions below.

EXAMPLE OUTPUTS

- **Example 1:** Prompt: "*A giraffe reading a book in a library, digital art style*" — produces a whimsical, colourful image of a giraffe with glasses holding an open book, surrounded by bookshelves. Shows good understanding of combining subject + action + setting + style.
- **Example 2:** Prompt: "*A futuristic matatu flying over Nairobi at sunset, in the style of a comic book*" — produces a vibrant, localized image blending familiar Kenyan context with sci-fi elements. Demonstrates creative combination of local culture and imaginative "what if" thinking.
- **Example 3 (comparison set):** Same subject ("a cat drinking tea") generated in four styles — realistic photo, watercolour painting, pixel art, and pencil sketch — shown side by side. Demonstrates understanding that style keywords significantly change the output while the subject stays recognizable.

FACILITATION TIPS

- **Pacing:** Spend no more than 10 minutes on the live demo — students learn fastest by trying prompts themselves. Reserve at least 25–30 minutes for hands-on generation.
- **Free-tier limits:** Warn students in advance that they have a limited number of image generations. Encourage them to plan/write their prompt in a doc first, then generate, rather than generating impulsively and running out of credits.
- **Common confusion:** Students often expect the AI to read their minds. Reassure them that vague prompts produce vague or strange results — this is a teaching moment about clear communication, not an AI failure.

- **Handling odd or "wrong" outputs:** If the AI produces something inaccurate (e.g., wrong number of fingers, garbled text, mismatched objects), use it as a discussion point rather than an error to hide — ask "why do you think that happened?"
- **Sensitive content:** Remind students that these tools have content filters and will block certain prompts (violence, real people, etc.). If a prompt is blocked, ask students to rephrase rather than try to bypass the filter.
- **Grouping:** Pairing students on one device can reduce the load on free-tier limits and encourages discussion about word choice.
- **Internet issues:** Have Craiyon (no login, browser-based) ready as backup if the main tool is slow or blocked by school firewalls.

DISCUSSION QUESTIONS

- What is a "prompt," and why does the wording of a prompt matter so much to the AI's output?
- How do you think the AI "knows" what a lion, a mango tree, or a watercolour style looks like?
- Can you think of a time today when the AI misunderstood your prompt? What might have caused that?
- Is the image the AI created "art"? Who do you think should be credited as the artist — you, the AI, or both? Why?
- AI image generators are trained on millions of real images made by human artists. What are the benefits and possible concerns with that?

EXTENSION CHALLENGES

- **Easy:** Generate the same subject in 3 different art styles (e.g., cartoon, realistic, watercolour) and write one sentence comparing the results.
- **Medium:** Create a short 4-image "story" using prompts that build on each other (e.g., a character's journey from morning to night), keeping the character consistent across prompts.
- **Hard:** Design a prompt for an image that represents an abstract idea (e.g., "friendship," "hope," or "climate change") without directly naming it in an obvious way, then present the image to the class and have them guess what abstract idea it represents.

Materials — Scaling

Each topic's materials list above is **per group**. Multiply it by the number of groups that run the topic (shown on each card) to get the totals to prepare. Shared kit (laptops, power strips, extension leads) can rotate between groups rather than be bought per group.

Safety, Logistics & Contingencies

- **Space:** one table per group with room to move; keep walkways clear and power leads taped down.
- **Movement:** groups start and stop together on the master timetable — stagger the break and lunch queues so the space never crowds.
- **First aid & allergies:** collect allergy info at registration; the first-aider holds the kit and the contact list all day.
- **Wet weather:** move outdoor free-play to the hall; keep one indoor back-up game ready.
- **Early finishers:** use each project's extension challenges (listed in the project pages below).
- **Equipment failure:** keep one spare kit per band; a failed build becomes a debugging lesson, not a lost hour.

Staff Briefing Checklist

- Every facilitator knows their group, their band, and their room.
- Kits counted and laid out per group before doors open.
- Registration desk has the sign-in sheet, badges, and allergy list.
- Timekeeper owns the master timetable and calls transitions out loud.
- Floating helpers know which groups they cover during breaks.
- Showcase space and closing certificates ready before the last block.